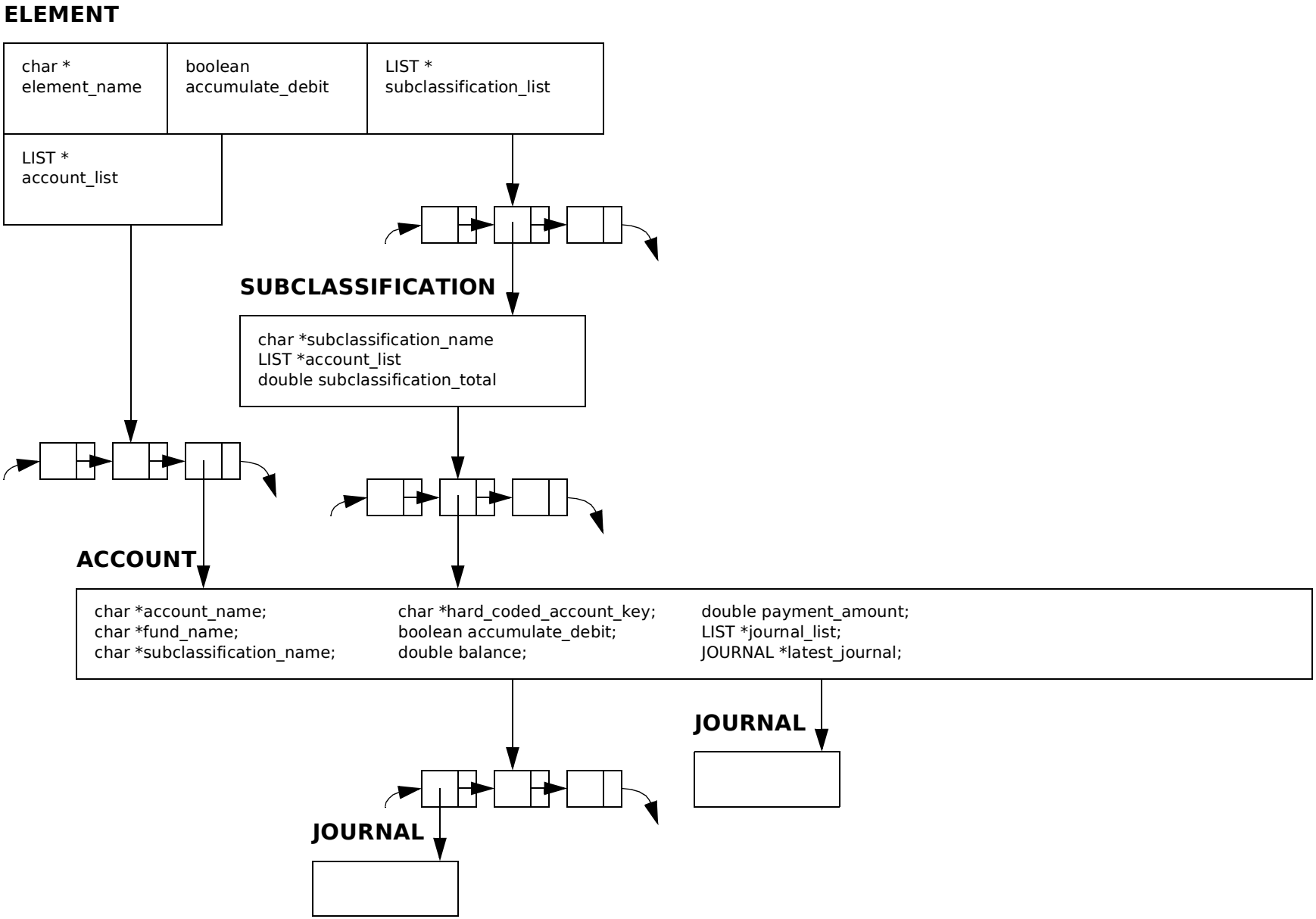
PredictiveBooks Data Structure



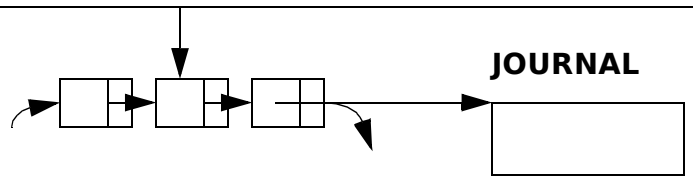
Tim Riley © 2012-2020

JOURNAL

```
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double previous_balance;
double previous_balance_database;
double debit_amount;
double credit_amount;
double balance;
double balance_database;
int transaction_count;
int transaction_count_database;
boolean match_sum_taken;

JOURNAL *journal_new(
        full_name,
        street_address,
        transaction_date_time,
        account_name );

char *journal_select(
        void );

JOURNAL *journal_parse(
        input );

LIST *journal_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_list_fetch(
        where_clause );

JOURNAL *journal_prior(
        transaction_date_time,
        account_name );

FILE *journal_insert_open(
        void );

void journal_insert(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        account_name,
        amount,
        is_debit );

/* Also does journal_propagate() for each account */
void journal_delete(
        full_name,
        street_address,
        transaction_date_time );

void journal_propagate(
        transaction_date_time,
        account_name );

LIST *journal_list_minimum(
        minimum_transaction_date_time,
        account_name );

LIST *journal_list_account(
        account_name );

/* Executes journal_list_set() */
LIST *journal_list_prior(
        prior_journal,
        account_name );

void journal_list_set(
        transaction_date_time,
        journal_list_prior() );

void journal_list_set_balances(
        journal_list_prior(),
        account_accumulate_debit(
            account_name ) );

void journal_list_propagate_update(
        journal_list_prior() );

JOURNAL *journal_account_fetch(
        account_name,
        transaction_date_time );

LIST *journal_account_name_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_binary_list(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        debit_account,
        credit_account );
```

TRANSACTION 1

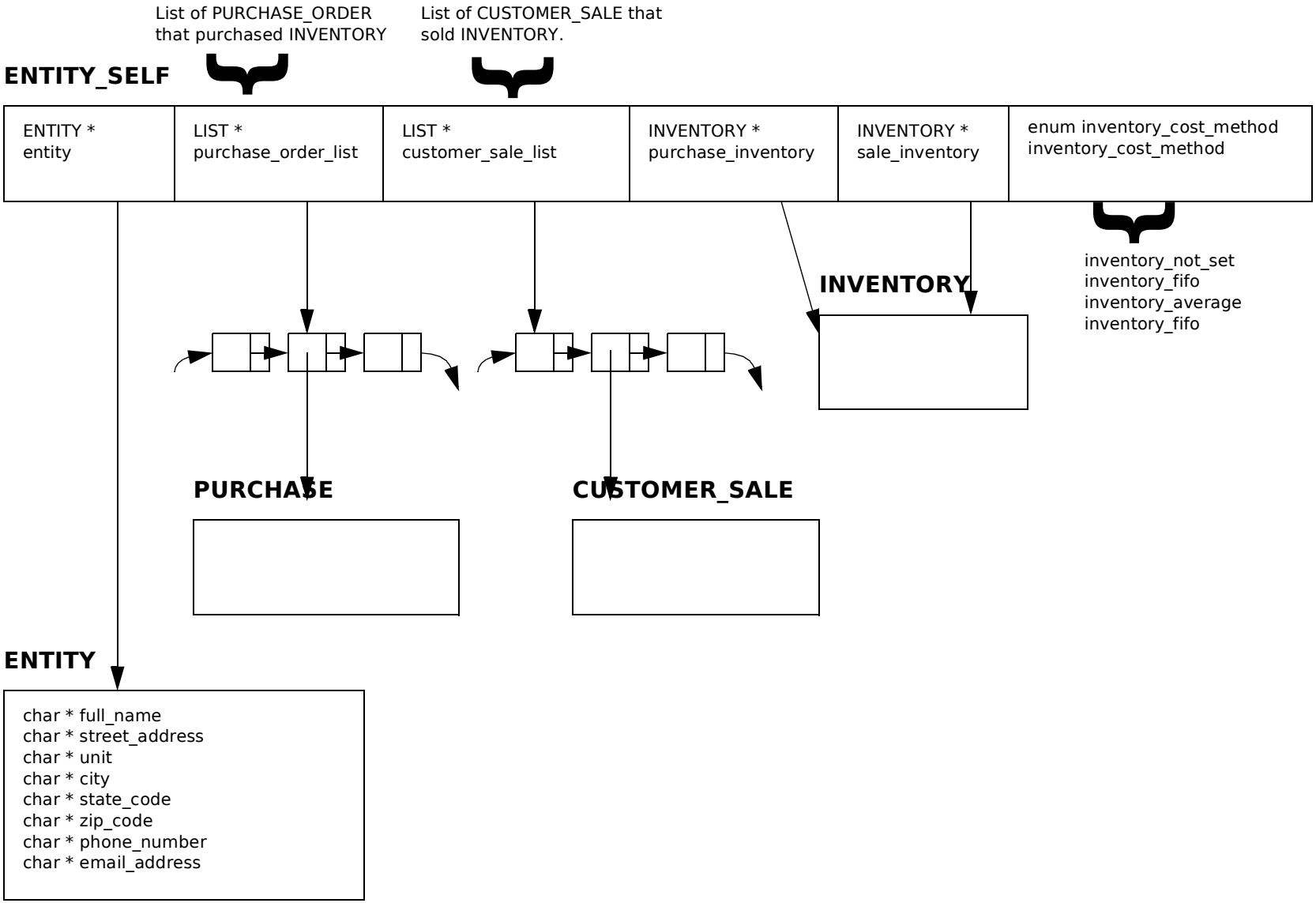
```
char *full_name;
char *street_address;
char *transaction_date_time;
double transaction_amount;
char *memo;
int check_number;
boolean lock_transaction;
char *property_street_address;
char *program_name;
LIST *journal_list;

TRANSACTION *transaction_calloc(
        void );

TRANSACTION *transaction_new(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_fetch(
        full_name,
        street_address,
        transaction_date_time );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

void transaction_refresh(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

/* Returns inserted transaction_date_time */
char *transaction_insert(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction );

FILE *transaction_insert_pipe_open(
        void );

/* Returns inserted transaction_date_time */
char *transaction_insert_pipe(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

void transaction_journal_list_insert(
        full_name,
        street_address,
        transaction_date_time,
        journal_list );

/* Returns transaction_list */
LIST *transaction_list_insert(
        transaction_list );

void transaction_list_journal_insert(
        transaction_list );

void transaction_update(
        transaction_amount,
        full_name,
        street_address,
        transaction_date_time );

char *transaction_select(
        void );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_parse(
        input_buffer );

/* Performs journal_propagate() */
void transaction_delete(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_seek(
        transaction_list,
        full_name,
        street_address,
        transaction_date_time );

/* Returns race-free transaction_date_time */
char *transaction_race_free(
        transaction_date_time );
```



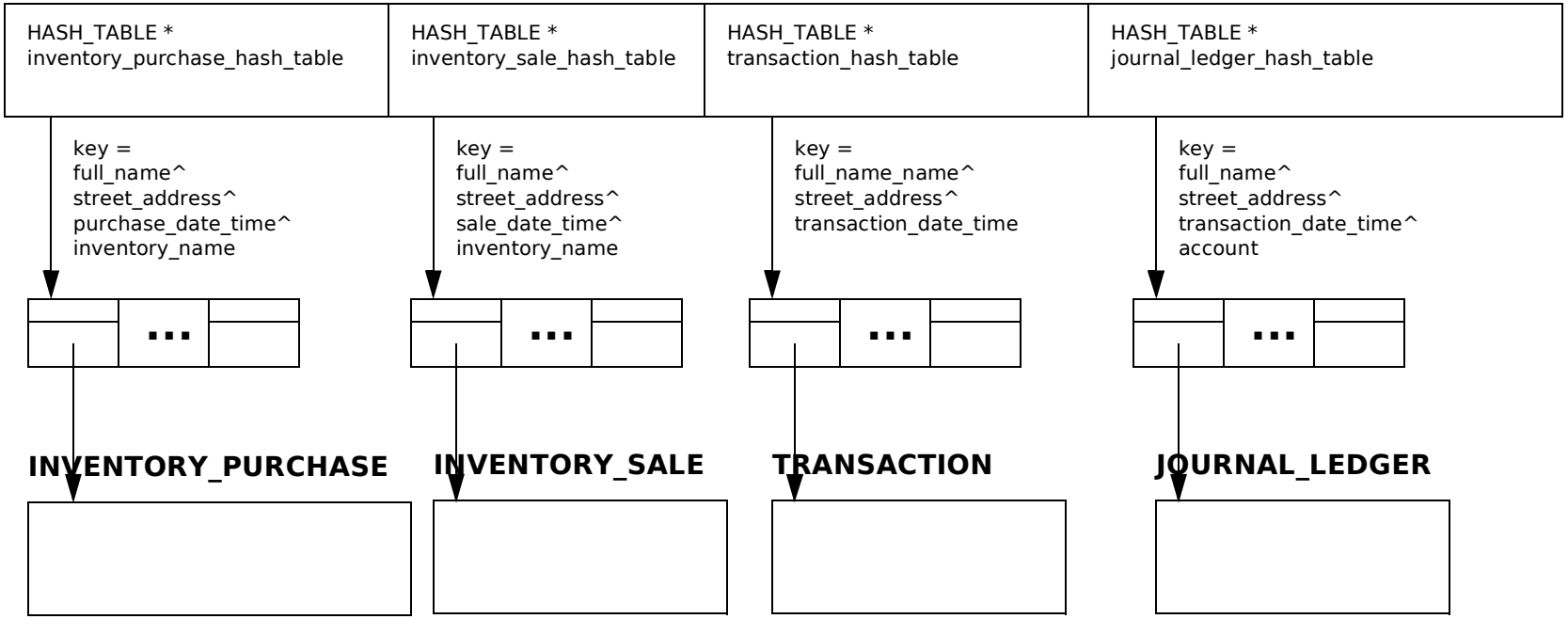
TRANSACTION 2

```
TRANSACTION *transaction_binary(
        full_name,
        street_address,
        transaction_date_time,
        char *debit_account,
        char *credit_account,
        double transaction_amount,
        char *memo );

LIST *transaction_binary_journal_list(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        debit_account,
        credit_account );

char *transaction_memo(
        memo );

void transaction_check_insert(
        insert_pipe,
        check_number );

void transation_lock_insert(
        insert_pipe,
        lock_transaction );

boolean transaction_exists(
        transaction_date_time );

char *transaction_display(
        transaction );

char *transaction_escape_full_name(
        full_name );

TRANSACTION *transaction_check_seek(
        transaction_list,
        check_number );
```

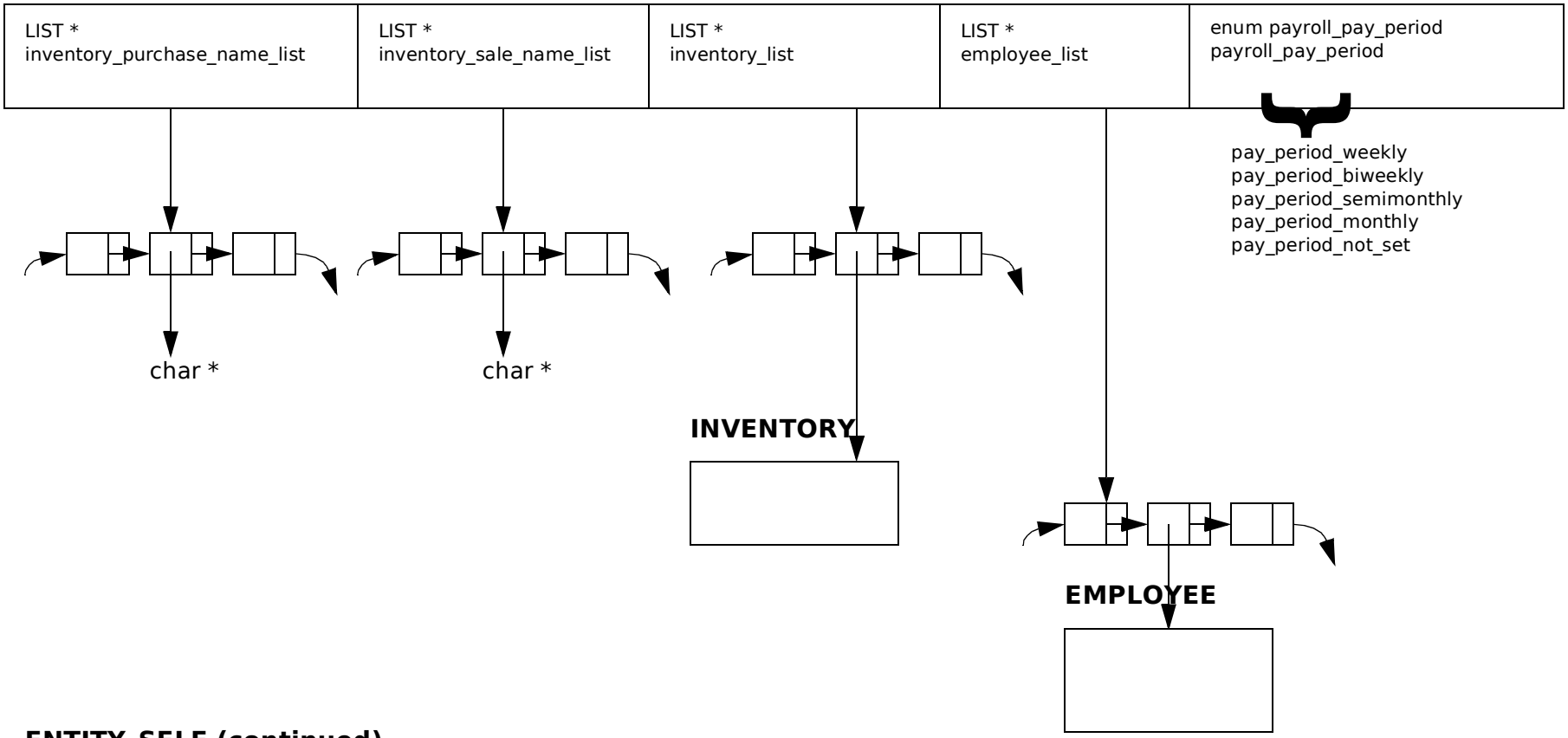
PredictiveBooks Data Structure



ENTITY_SELF (continued)



ENTITY_SELF (continued)



ENTITY_SELF (continued)

char *payroll_beginning_day	int federal_unemployment_wage_base
double social_security_combined_tax_rate	double federal_unemployment_tax_standard_rate
int social_security_payroll_ceiling	double federal_unemployment_threshold_rate
double medicare_combined_tax_rate	double federal_unemployment_tax_minimum_rate
double medicare_additional_withholding_rate	int state_unemployment_wage_base
int medicare_additional_gross_pay_floor	double state_unemployment_tax_rate
double federal_withholding_allowance_period_value	double state_sales_tax_rate
double federal_nonresident_withholding_income_premium	
double state_withholding_allowance_period_value	
double state_itemized_allowance_period_value	

CUSTOMER_SALE (1)

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *completed_date_time;
double shipping_revenue;
enum title_passage_rule title_passage_rule;
double sales_tax_database;
char *transaction_date_time_database;

char *customer_sale_primary_where(
        full_name,
        street_address,
        sale_date_time );

double customer_sale_gross_revenue(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total() );

double customer_sale_sales_tax(
        inventory_sale_list(),
        entity_self_sales_tax_rate() );

double customer_sale_invoice_amount(
        customer_sale_gross_revenue(),
        customer_sale_sales_tax(),
        shipping_charge );

double Customer_sale_amount_due(
        customer_sale_invoice_amount(),
        customer_payment_total() )
{
        return A - B;
}

TRANSACTION *customer_sale_transaction(
        full_name,
        street_address,
        sale_date_time,
        invoice_amount,
        gross_revenue,
        sale_tax,
        shipping_charge,
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
        account_sales_tax_payable() );

void customer_sale_update(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total(),
        customer_sale_gross_revenue()
        customer_sale_sales_tax(),
        customer_sale_invoice_amount(),
        customer_payment_total(),
        customer_sale_amount_due(),
        customer_sale_transaction()->
            transaction_date_time,
        full_name,
        street_address,
        sale_date_time );

CUSTOMER_SALE *customer_sale_steady_state(
        full_name,
        street_address,
        sale_date_time,
        inventory_sale_list(),
        fixed_service_sale_list(),
        hourly_service_sale_list(),
        customer_payment_list(),
        shipping_charge,
        entity_self_sales_tax_rate(),
        sales_tax_database,
        completed_date_time,
        transaction_date_time_database.
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
```

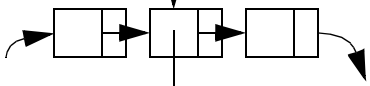
TRANSACTION

ENTITY

FOB_shipping
FOB_destination
title_passage_rule_null

CUSTOMER_SALE (2)

```
LIST *customer_payment_list(
        full_name,
        street_address,
        sale_date_time );
```



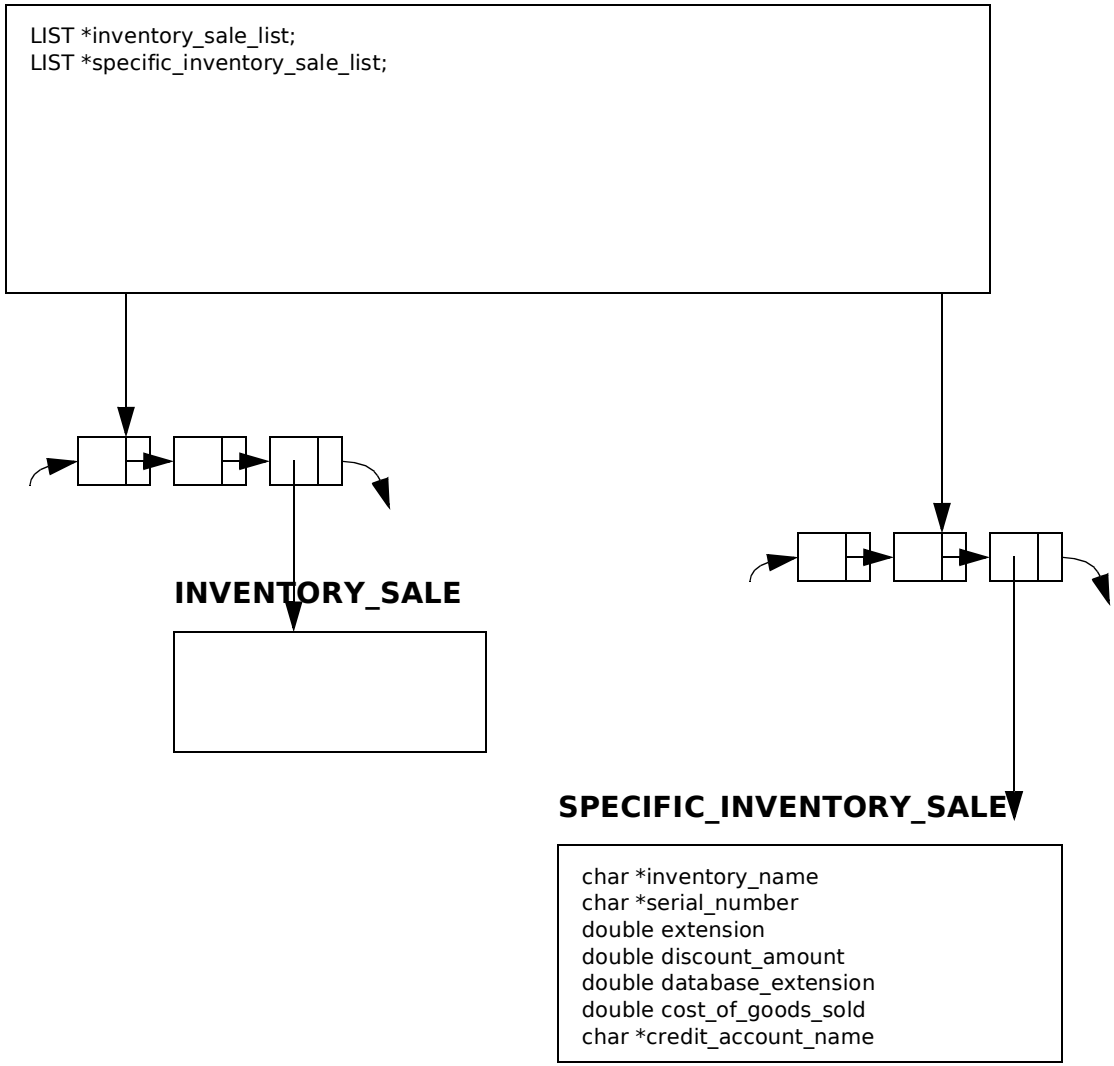
CUSTOMER_PAYMENT

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

double customer_payment_total(
        customer_payment_list() )
{
        return sum( A->payment_amount );
}

CUSTOMER_PAYMENT *customer_payment_new(
        full_name,
        street_address,
        sale_date_time,
        payment_date_time );

char *customer_payment_select(
        void );

char *customer_payment_list_fetch(
        where );

CUSTOMER_PAYMENT *customer_payment_parse(
        input );

LIST *customer_payment_system_list(
        sys_string );

TRANSACTION *customer_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        account_cash(),
        account_receivable() );
```

ENTITY

TRANSACTION

CUSTOMER_SALE (3)



PredictiveBooks Data Structure

HOURLY_SERVICE_SALE

```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
char *service_description;
double hourly_rate;
double discount_amount;
double work_hours_database;
int estimated_hours;
double net_revenue_database;

char *hourly_service_sale_revenue_account_name(
        service_name,
        service_description );

LIST *hourly_service_work_list(
        full_name,
        street_address,
        sale_date_time,
        service_description );

double hourly_service_work_hours =
        work_list_hours(
            hourly_service_work_list() );

double hourly_service_sale_net_revenue(
        hourly_rate,
        hourly_service_work_hours,
        discount_amount )
{
    return (A x B) - C;
}
```

CUSTOMER_SALE (4)

```
LIST *    fixed_service_sale_list
LIST *    hourly_service_sale_list
```

FIXED_SERVICE_SALE

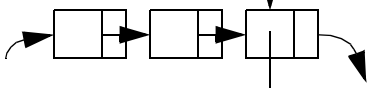
```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
double fixed_price;
int estimated_hours;
double discount_amount;
double work_hours_database;
double net_revenue_database;

LIST *fixed_service_work_list(
        full_name,
        street_address,
        sale_date_time,
        service_name );

char *fixed_service_sale_revenue_accouint_name(
        service_name );

double fixed_service_sale_work_hours =
        work_list_hours(
            fixed_service_work_list() );

double fixed_service_sale_net_revenue(
        fixed_price,
        discount_amount )
{
    return A - B;
}
```

WORK

```
ENTITY *customer_entity = entity_new();
char *sale_date_time;
char *service_name;
char *begin_date_time;
char *end_date_time;
char *login_name;
char *service_description;
char *activity_name;

double work_hours(
        begin_date_time,
        end_date_time );

double work_list_hours(
        work_list );
```

PURCHASE (1)

```
ENTITY *vendor_entity =
        entity_fetch(
            full_name,
            street_address );

char *purchase_date_time;
double sales_tax;
double freight_in;
char *arrived_date_time;
char *program_name;
char *property_street_address;

double purchase_equipment_total(
        purchase_equipment_list() )
{
        return sum(
                A->
                quipment_purchase->
                purchase_price );
}

double Purchase_invoice_amount(
        purchase_equipment_total(),
        sales_tax,
        freight_in );

double purchase_fetch_amount_due(
        full_name,
        street_address,
        purchase_date_time );

double Purchase_amount_due(
            Purchase_invoice_amount(),
            vendor_payment_total() )
{
        return A - B;
}

LIST *purchase_equipment_list(
        full_name,
        street_address,
        purchase_date_time );

TRANSACTION *purchase_transaction(
        full_name,
        street_address,
        arrived_date_time,
        Purchase_invoice_amount(),
        purchase_asset_account_name(
            purchase_equipment_list() ),
        account_payable() );

void purchase_update(
        purchase_equipment_total(),
        Purchase_invoice_amount(),
        vendor_payment_total(),
        Purchase_amount_due(),
        purchase_transaction()->
            transaction_date_time,
        full_name,
        street_address,
        purchase_date_time );

/* Returns true transaction_date_time */
char *purchase_transaction_refresh(
        purchase_transaction()->
            transaction_amount,
        purchase_transaction()->
            journal_list,
        purchase_transaction->memo,
        full_name,
        street_address,
        purchase_transaction()->
            transaction_date_time );

LIST *purchase_journal_list(
        Purchase_invoice_amount(),
        purchase_asset_account_name(),
        account_payable() );
```



EQUIPMENT_PURCHASE

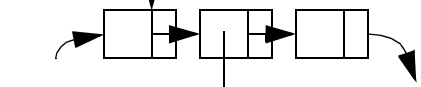
```
char *asset_name;
char *serial_number;
ENTITY *vendor_entity;
char *purchase_date_time;
char *service_placement_date;
double equipment_cost;
char *asset_account_name;
LIST *equipment_depreciation_list;
```

ENTITY

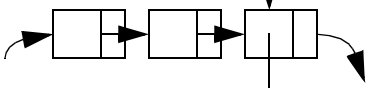
```
char *full_name;
char *street_address;
```

PURCHASE (2)

```
double   sum_inventory_extension
double   sum_specific_inventory_extension
double   sum_supply_extension
double   sum_service_extension
double   sum_fixed_asset_extension
double   sum_prepaid_asset_extension
doulbe   sum_extension
double   database_sum_extension
double   sales_tax
double   freight_in
double   purchase_amount
double   sum_payment_amount

double   amount_due
double   database_amount_due
double   liability_payment_amount
char *   shipped_date
char *   database_shipped_date
char *   arrived_date_time
char *   database_arrived_date_time
LIST *   inventory_purchase_list
LIST *   specific_inventory_purchase_list
LIST *   service_purchase_list
LIST *   supply_purchase_list
LIST *   property_fixed_asset_purchase_list

LIST *   prepaid_asset_purchase_list

TRANSACTION *purchase_transaction(
         full_name,
         street_address,
         arrived_date_time,
         purchase_invoice_amount,
         purchase_asset_account_name(
              purchase_equipment_list() ),
         account_payable() );

char *property_street_address
```



INVENTORY_PURCHASE

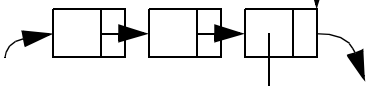


PURCHASE_SUPPLY

```
char *supply_name
int quantity
double unit_cost
double extension
double database_extension
char *account_name
```

```
enum title_passage_rule
title_passage_rule
```

FOB_shipping
FOB_destination
title_passage_rule_null

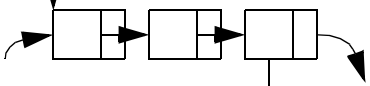


PURCHASE_SERVICE

```
char *account_name
double extension
```

PURCHASE_SPECIFIC_INVENTORY

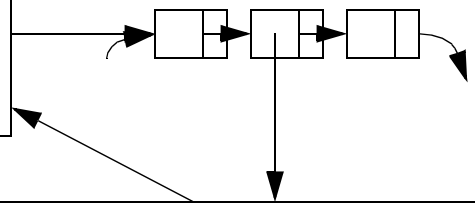
```
char *inventory_name
char *serial_number
double extension
double capitalized_extension
double database_capitalized_extension
```



PURCHASE_PREPAID_ASSET

PURCHASE (3)

```
LIST *purchase_vendor_payment_list =
        vendor_payment_list(
            full_name,
            street_address,
            purchase_date_time );
```



VENDOR_PAYMENT

```
ENTITY *vendor_entity =
        entity_fetch(
            full_name,
            street_address );

char *purchase_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

PURCHASE *purchase;

LIST *vendor_payment_list(
        full_name,
        street_address,
        purchase_date_time );

LIST *vendor_payment_fetch_list(
        where );

VENDOR_PAYMENT *vendor_payment_new(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

VENDOR_PAYMENT *vendor_payment_fetch(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

char *vendor_payment_select();

char *vendor_payment_primary_where(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

VENDOR_PAYMENT *vendor_payment_parse(
        input );

LIST *vendor_payment_system_list(
        sys_string );

double vendor_payment_total(
        LIST *vendor_payment_list() )
{
        return sum(
                A->
                vendor_payment->
                payment_amount );
}

TRANSACTION *vendor_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

LIST *vendor_payment_journal_list(
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

void vendor_payment_insert(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time,
        payment_amount,
        check_number );

VENDOR_PAYMENT *vendor_payment_seek(
        vendor_payment_list,
        payment_date_time );

char *vendor_payment_update_sys_string(
        void );
```

EQUIPMENT_PURCHASE (1)

```
char *    asset_name;
char *    serial_number;
char *    service_placement_date;
double    equipment_cost;
int       estimated_useful_life_years;
int       estimated_useful_life_units;
int       estimated_residual_value;
int       declining_balance_n;
char *    depreciation_method;
double    tax_cost_basis;
char *    tax_recovery_period;
char *    disposal_date;
char *    equipment_account_name;
double    finance_accumulated_depreciation;
double    tax_accumulated_depreciation;
TAX_RECOVERY *tax_recovery;

ENTITY *vendor_entity =
          vendor_fetch(
               full_name,
               street_address );

EQUIPMENT_PURCHASE *
          equipment_purchase_new(
               asset_name,
               serial_number,
               full_name,
               street_address,
               purchase_date_time );

EQUIPMENT_PURCHASE *
          equipment_purchase_fetch(
               asset_name,
               serial_number,
               full_name,
               street_address,
               purchase_date_time );

EQUIPMENT_PURCHASE *
          equipment_purchase_parse(
               input );

char *equipment_purchase_select(
          void );

FILE *equipment_purchase_update_open(
          void );

void equipment_purchase_update(
          finance_accumulated_depreciation,
          tax_accumulated_depreciation,
          asset_name,
          serial_number,
          full_name,
          street_address,
          purchase_date_time );

LIST *equipment_purchase_list(
          full_name,
          street_address,
          purchase_date_time );
```

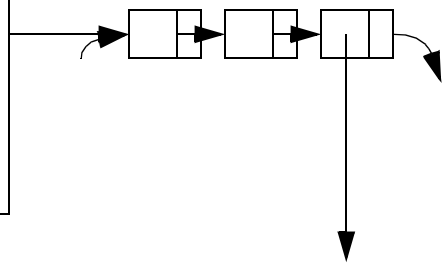
TAX_RECOVERY

char *	service_placecment_date	char *	disposal_date
double	tax_cost_basis	int	tax_year
char *	tax_recovery_period	double	recovery_amount
double	tax_recovery_period_years	double	recovery_percent

ENTITY

EQUIPMENT_PURCHASE (2)

```
LIST *depreciation_list =
        depreciation_list(
            asset_name,
            serial_number,
            full_name,
            street_address,
            purchase_date_time );

DEPRECIATION *equipment_purchase_depreciation(
        equipment_purchase,
        depreciation_date,
        prior_depreciation_date,
            transaction_date_time,
            units_produced );

LIST *equipment_purchase_depreciation_list(
        equipment_purchase_list );

/* Returns equipment_purchase_list with */
/* equipment_purchase->depreciation_list set. */
LIST *equipment_purchase_list_depreciate(
        equipment_purchase_list(),
        depreciation_date,
        boolean set_depreciation_date );
```



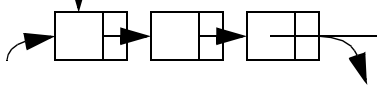
DEPRECIATION

```
char *asset_name;
char *serial_number;

ENTITY *vendor_entity =
        entity_new(
            full_name,
            street_address ) );

char *purchase_date_time;
char *depreciation_date;

double depreciation_accumulated_depreciation(
        prior_accumulated_depreciation,
        depreciation_amount() );

double depreciation_amount(
        depreciation_method,
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        estimated_useful_life_units,
        declining_balance_n,
        prior_depreciation_date,
        depreciation_date_string,
        finance_accumulated_depreciation,
        service_placement_date,
        units_produced );

double depreciation_straight_line(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation );

double depreciation_sum_of_years_digits(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation,
        service_placement_date );

double depreciation_double_declining_balance(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation );

double depreciation_n_declining_balance(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation,
        n );

double depreciation_units_of_production(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_units,
        units_produced,
        finance_accumulated_depreciation );

double depreciation_fraction_of_year(
        prior_depreciation_date,
        depreciation_date );

double depreciation_amount_total(
        depreciation_list() );

LIST *depreciation_list(
        asset_name,
        serial_number,
        full_name,
        street_address,
        purchase_date_time );

TRANSACTION *depreciation_transaction(
        full_name,
        street_address,
        transaction_date_time,
        depreciation_amount,
        account_depreciation_expense(),
        account_accumulated_depreciation() );

LIST *depreciation_transaction_list(
        depreciation_list );
```

PURCHASE_PREPAID_ASSET

char *	full_name	char *	asset_account_name	double	accumulated_accrual
char *	street_address	char *	expense_account_name		
char *	purchase_date_time	double	extension		
char *	asset_name	double	accrual_period_years		

LIST *
accrual_list



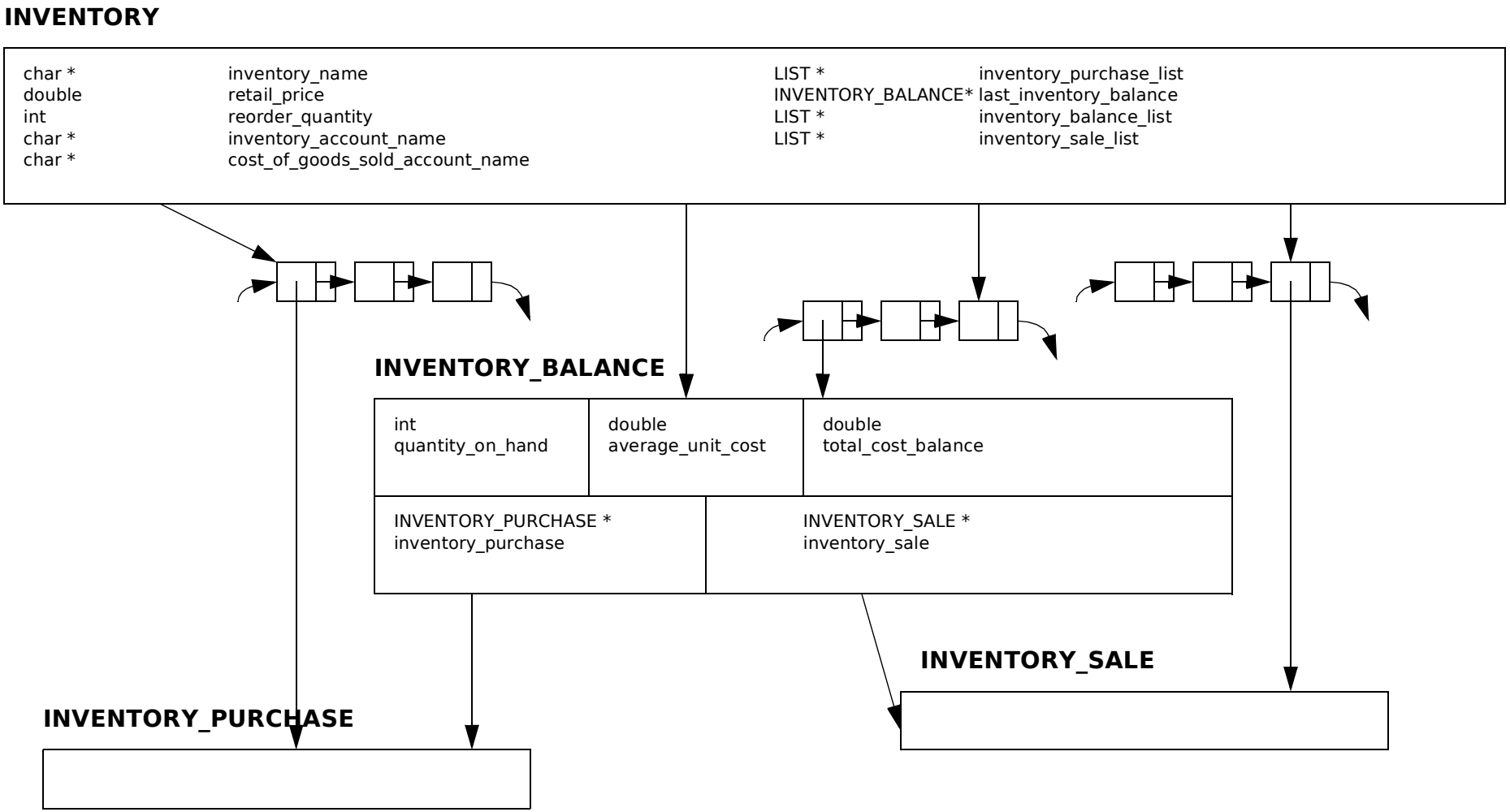
ACCRUAL

char *	full_name	double	accrual_amount
char *	street_address	double	database_accrual_amount
char *	purchase_date_time	char *	transaction_date_time
char *	asset_name	char *	database_transaction_date_time
char *	accrual_date	TRANSACTION *transaction	

VENDOR_PAYMENT

char * payment_date_time	double payment_amount	char * transaction_date_time	char * database_transaction_date_time	TRANSACTION * transaction

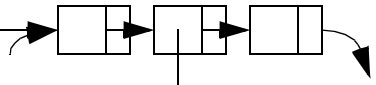
TRANSACTION


INVENTORY
char * inventory_name
double retail_price
int reorder_quantity
char * inventory_account_name
char * cost_of_goods_sold_account_name
LIST * inventory_purchase_list
INVENTORY_BALANCE* last_inventory_balance
LIST * inventory_balance_list
LIST * inventory_sale_list
INVENTORY_BALANCE
int quantity_on_hand
double average_unit_cost
double total_cost_balance
INVENTORY_PURCHASE * inventory_purchase
INVENTORY_SALE * inventory_sale
INVENTORY_SALE
INVENTORY_PURCHASE

PredictiveBooks Data Structure

INVENTORY_PURCHASE

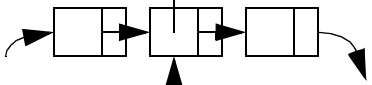
char *	full_name	int	database_arrived_quantity
char *	street_address	int	missing_quantity
char *	purchase_date_time	int	quantity_on_hand
char *	inventory_name	int	database_quantity_on_hand
int	ordered_quantity	int	layer_consumed_quantity
double	unit_cost	double	average_unit_cost
double	capitalized_unit_cost	double	database_average_unit_cost
double	database_capitalized_unit_cost	char *	arrived_date_time
double	extension	char *	inventory_account_name
double	database_extension	LIST *	inventory_purchase_return_list
int	arrived_quantity		



INVENTORY_PURCHASE_RETURN

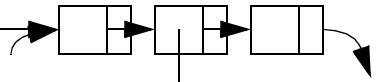
char *	return_date_time
int	returned_quantity
double	sales_tax
char *	transaction_date_time
char *	database_transaction_date_time
TRANSACTION *	transaction

TRANSACTION



INVENTORY_SALE

char *	full_name	double	database_extension
char *	street_address	double	cost_of_goods_sold
char *	sale_date_time	double	database_cost_of_goods_sold
char *	inventory_name	char *	completed_date_time
int	sold_quantity	char *	inventory_account_name
double	retail_price	char *	cost_of_goods_sold_account_name
double	discount_amount	LIST *	inventory_sale_return_list
double	extension	LIST *	layer_inventory_purchase_list

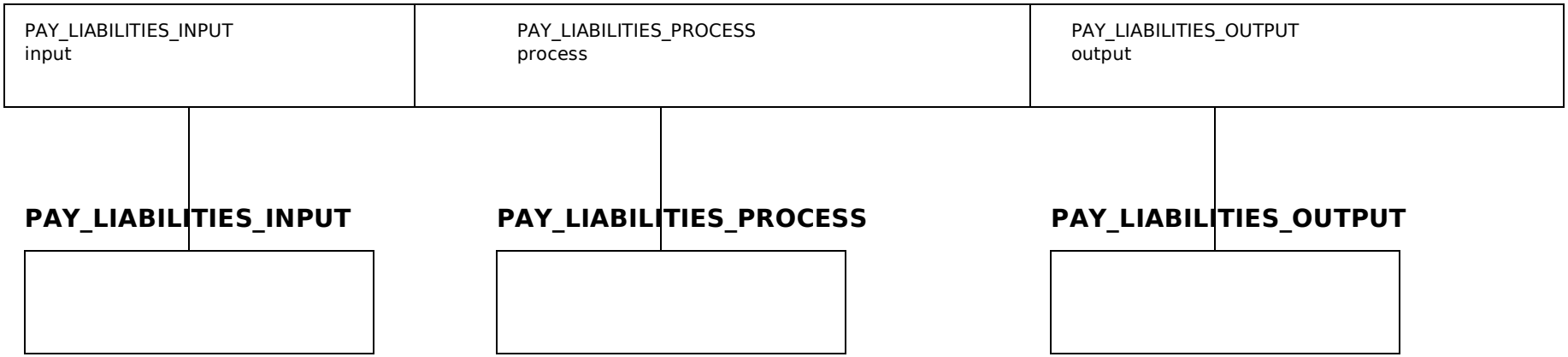


INVENTORY_SALE_RETURN

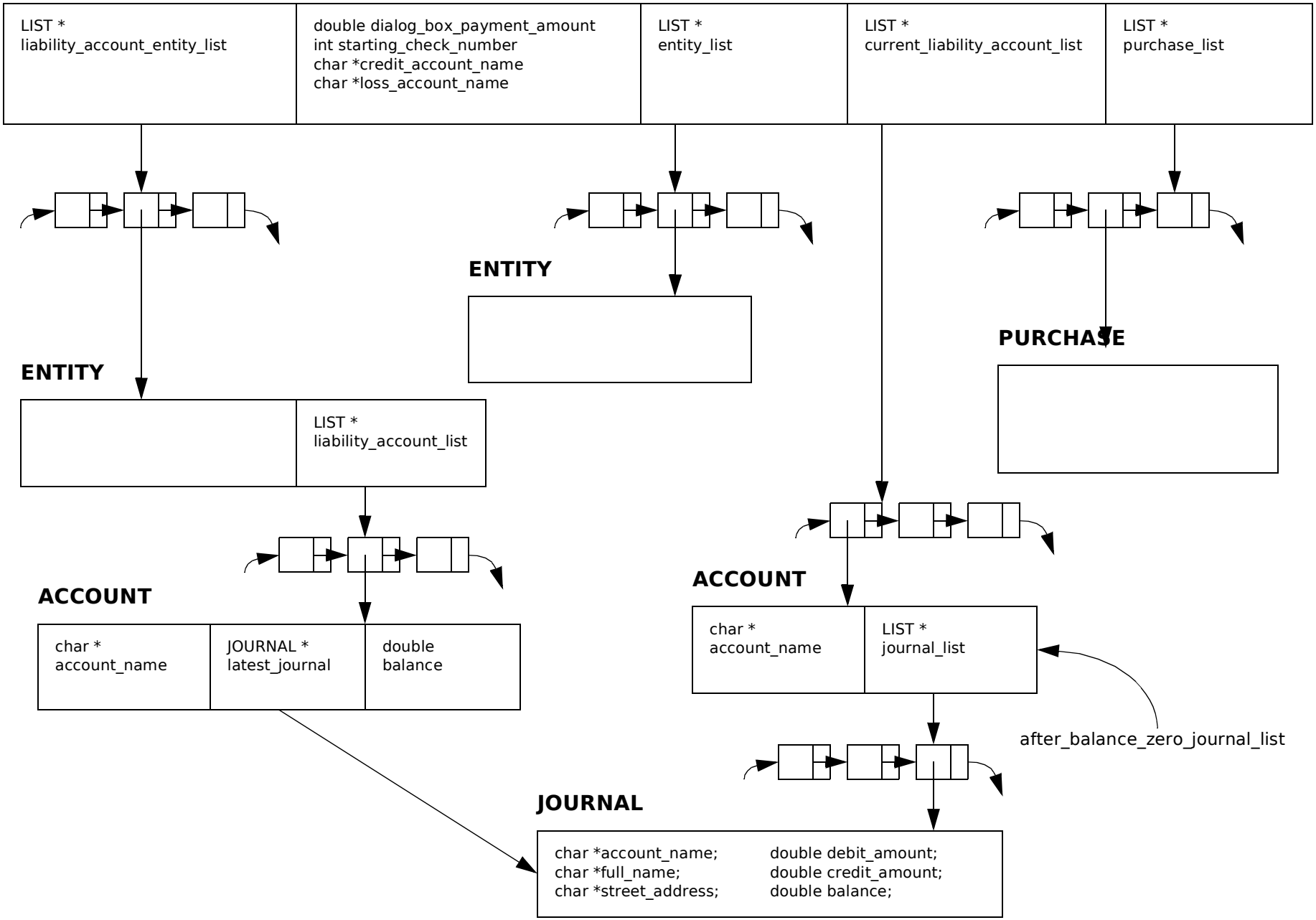
char *	return_date_time
int	returned_quantity
char *	transaction_date_time
char *	database_transaction_date_time
TRANSACTION *	transaction

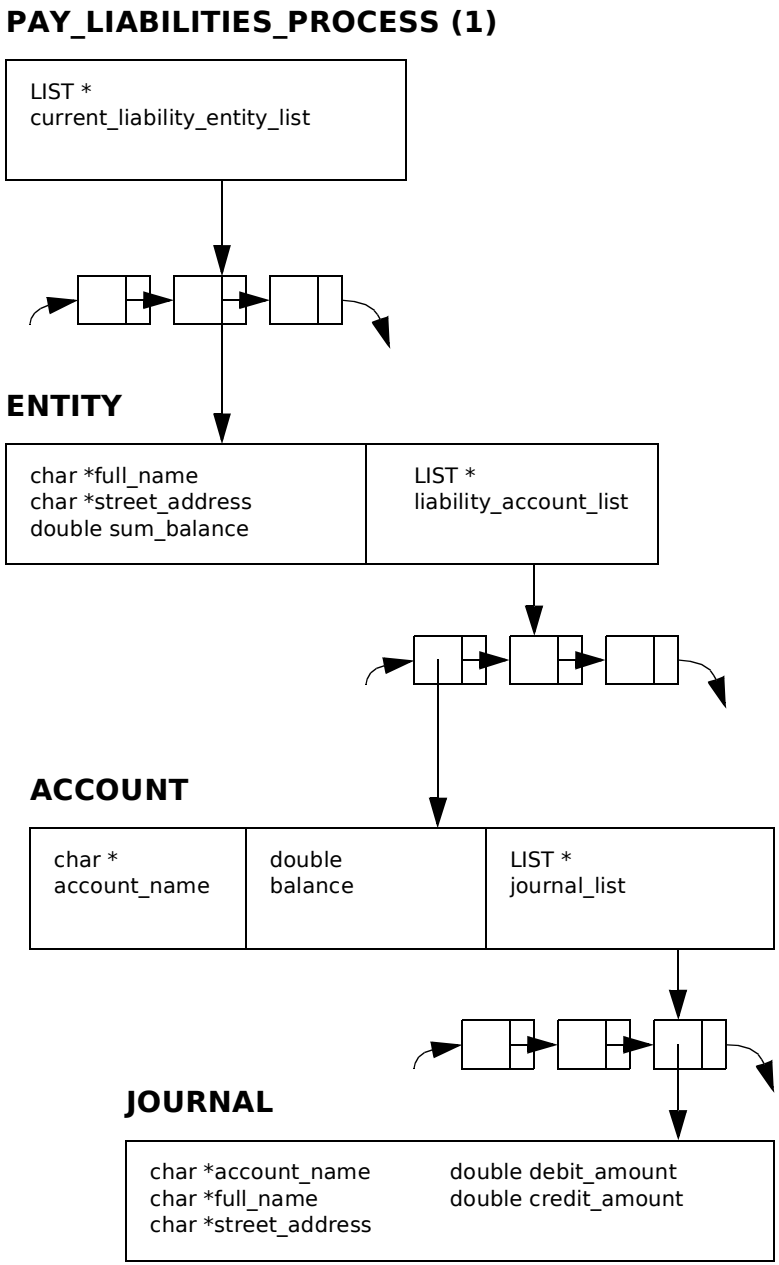
TRANSACTION

PAY_LIABILITIES

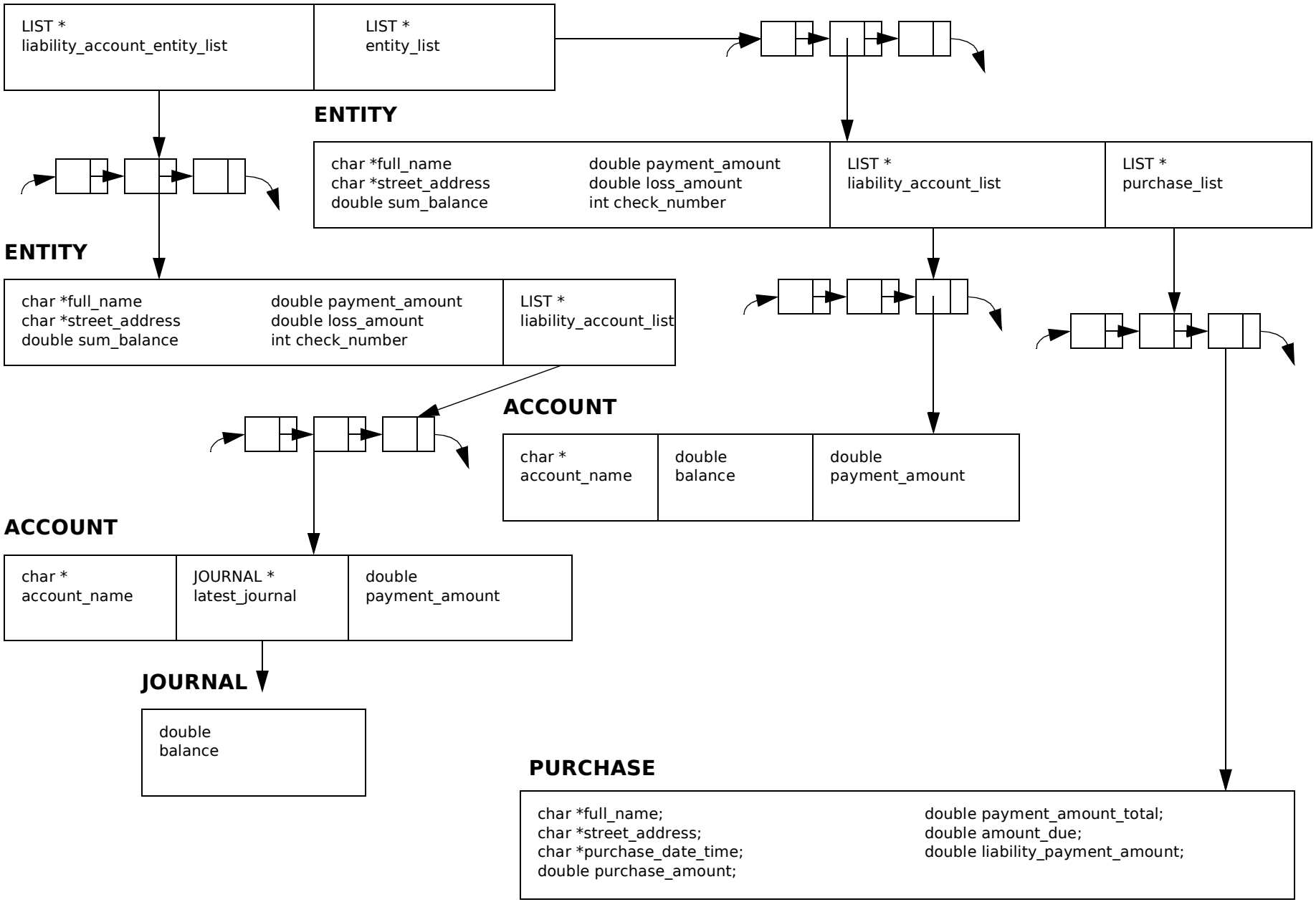


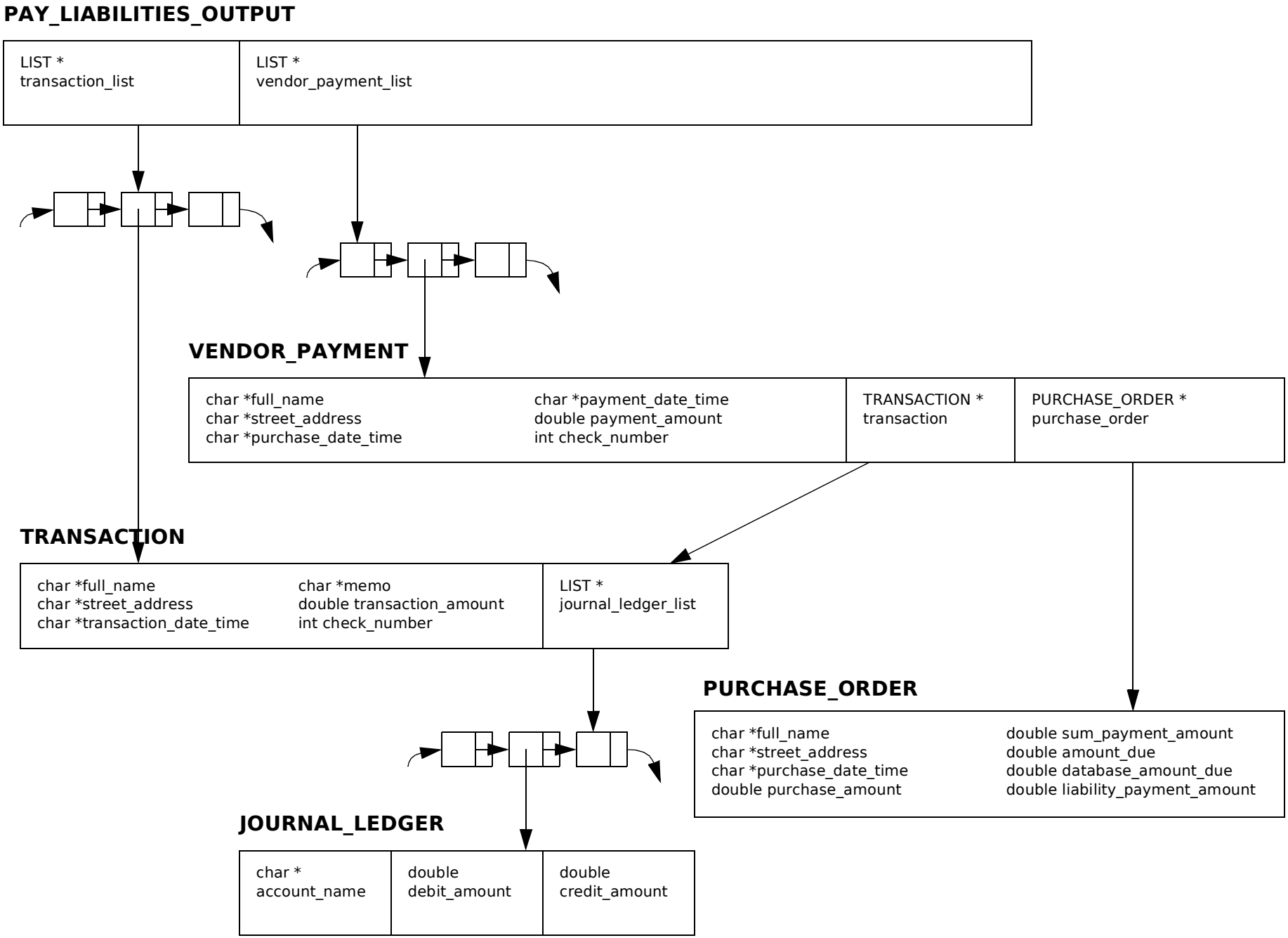
PAY_LIABILITIES_INPUT




PAY_LIABILITIES_PROCESS (1)
LIST *
current_liability_entity_list
ENTITY
char *full_name
char *street_address
double sum_balance
LIST *
liability_account_list
ACCOUNT
char *
account_name
double
balance
LIST *
journal_list
JOURNAL
char *account_name
char *full_name
char *street_address
double debit_amount
double credit_amount

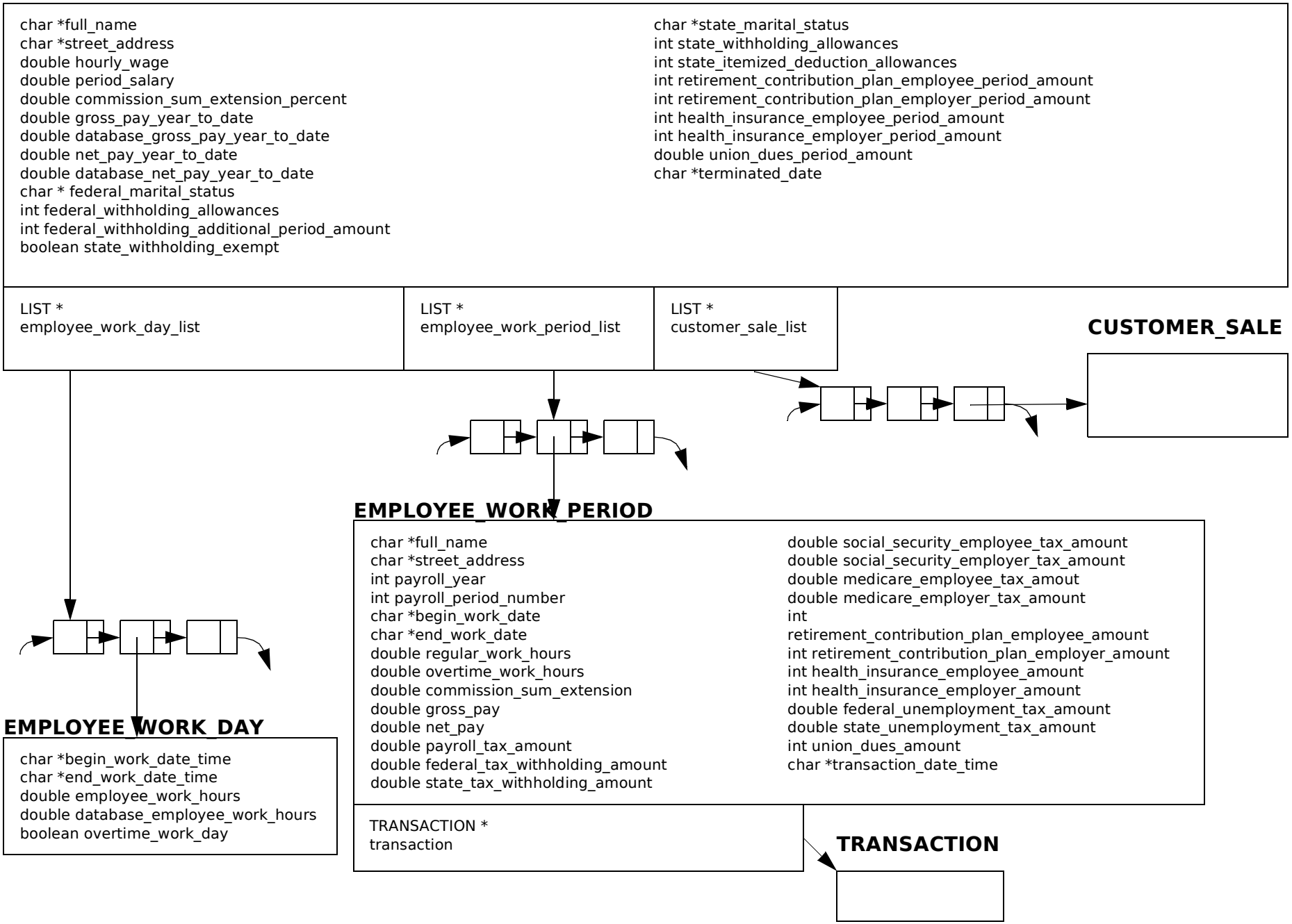
PAY_LIABILITIES_PROCESS (2)

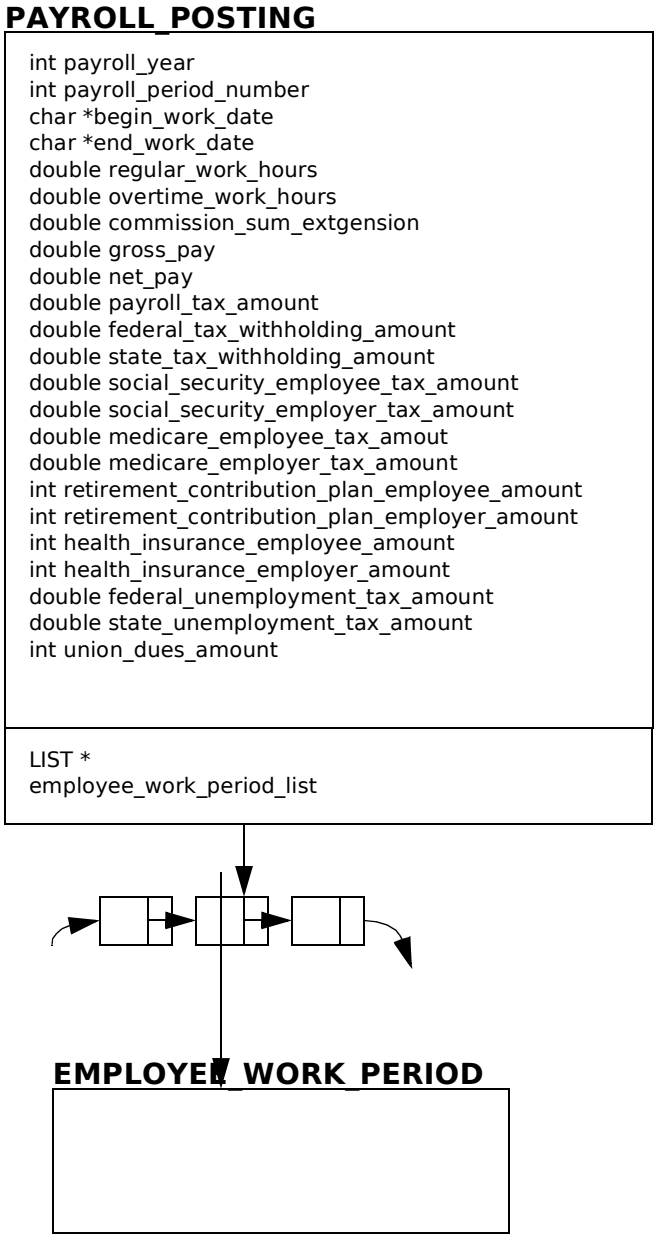


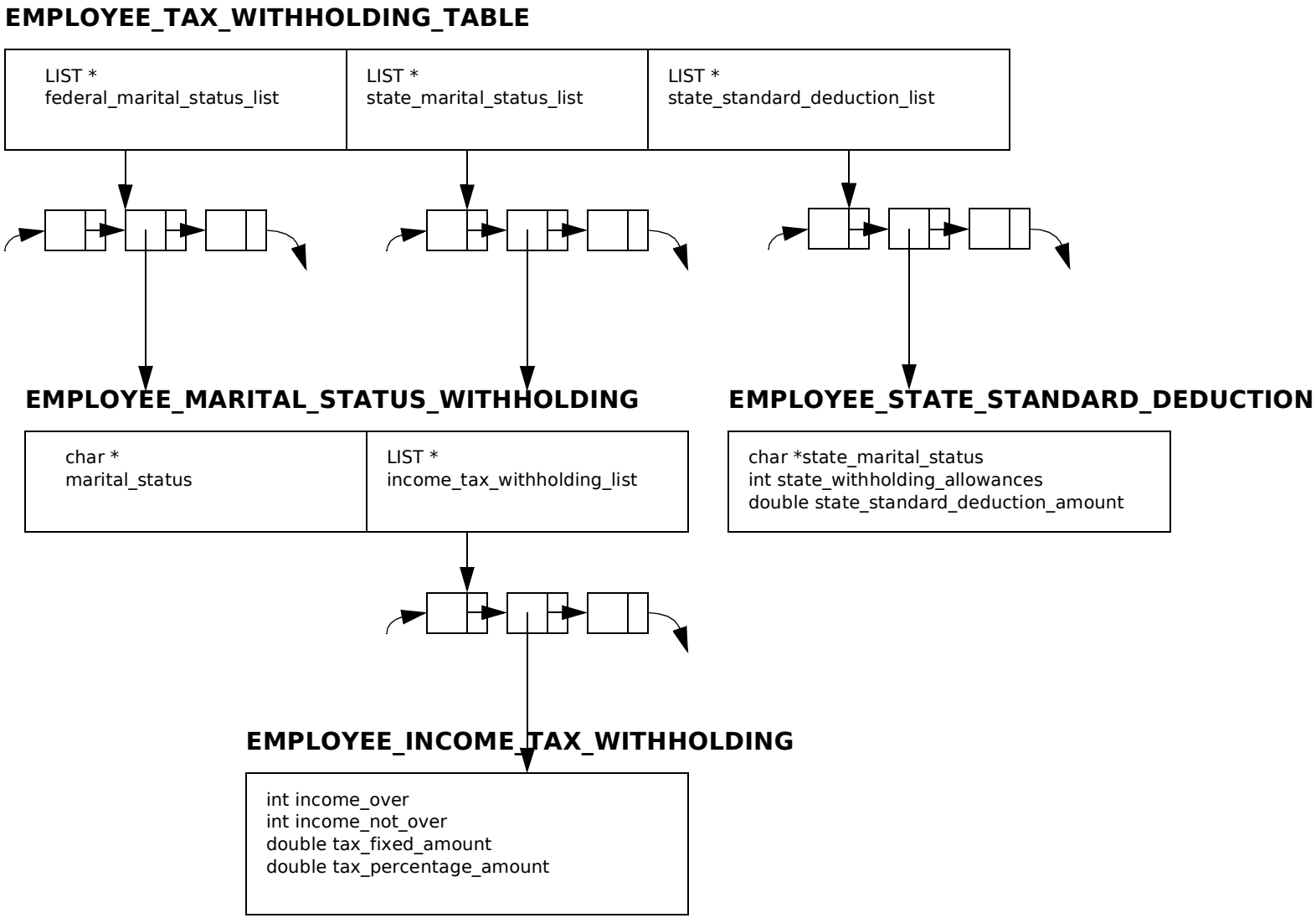

PAY_LIABILITIES_OUTPUT
LIST *
transaction_list
LIST *
vendor_payment_list
VENDOR_PAYMENT
char *full_name
char *street_address
char *purchase_date_time
char *payment_date_time
double payment_amount
int check_number
TRANSACTION *
transaction
PURCHASE_ORDER *
purchase_order
TRANSACTION
char *full_name
char *street_address
char *transaction_date_time
char *memo
double transaction_amount
int check_number
LIST *
journal_ledger_list
PURCHASE_ORDER
char *full_name
char *street_address
char *purchase_date_time
double purchase_amount
double sum_payment_amount
double amount_due
double database_amount_due
double liability_payment_amount
JOURNAL_LEDGER
char *
account_name
double
debit_amount
double
credit_amount

PredictiveBooks Data Structure

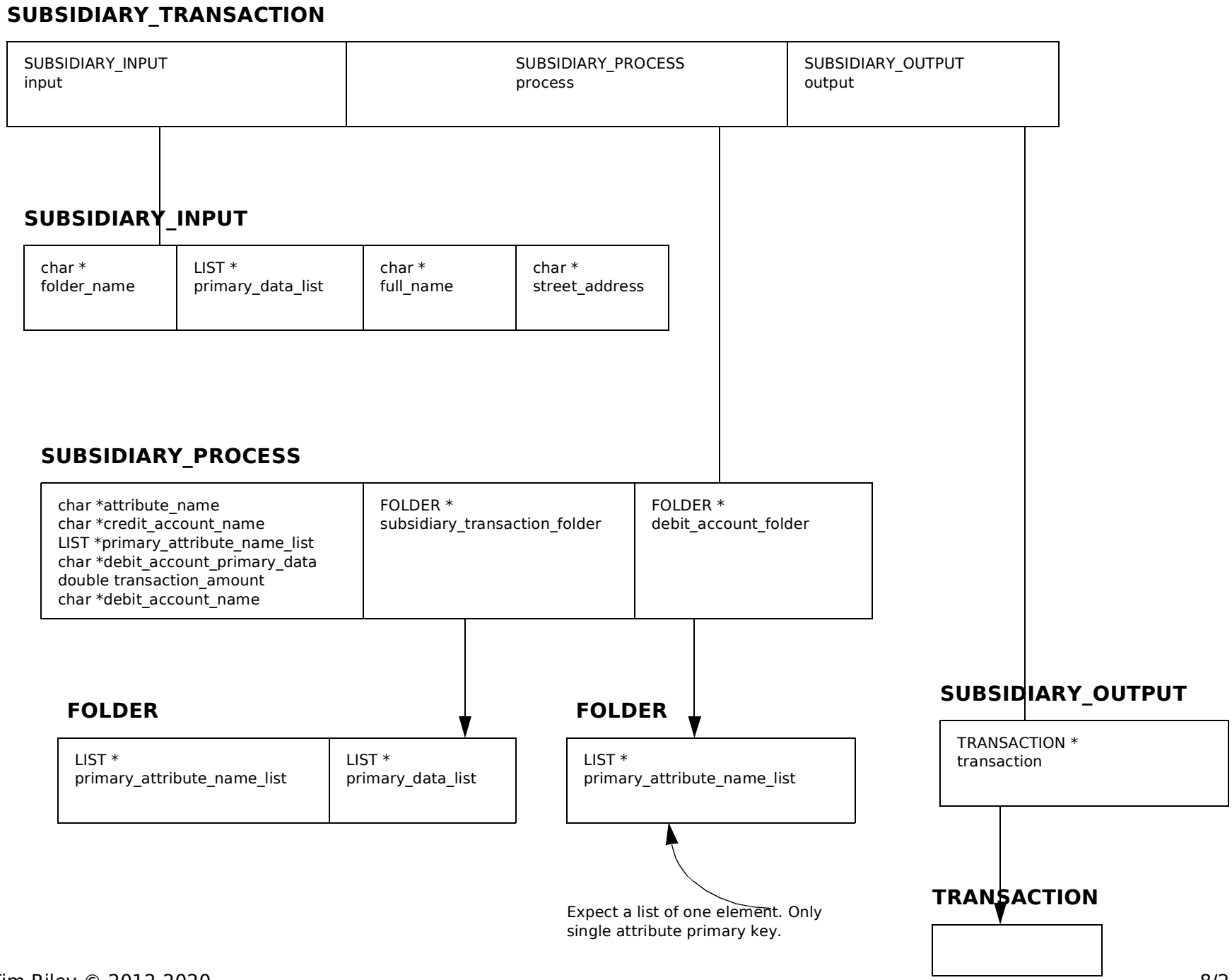
EMPLOYEE

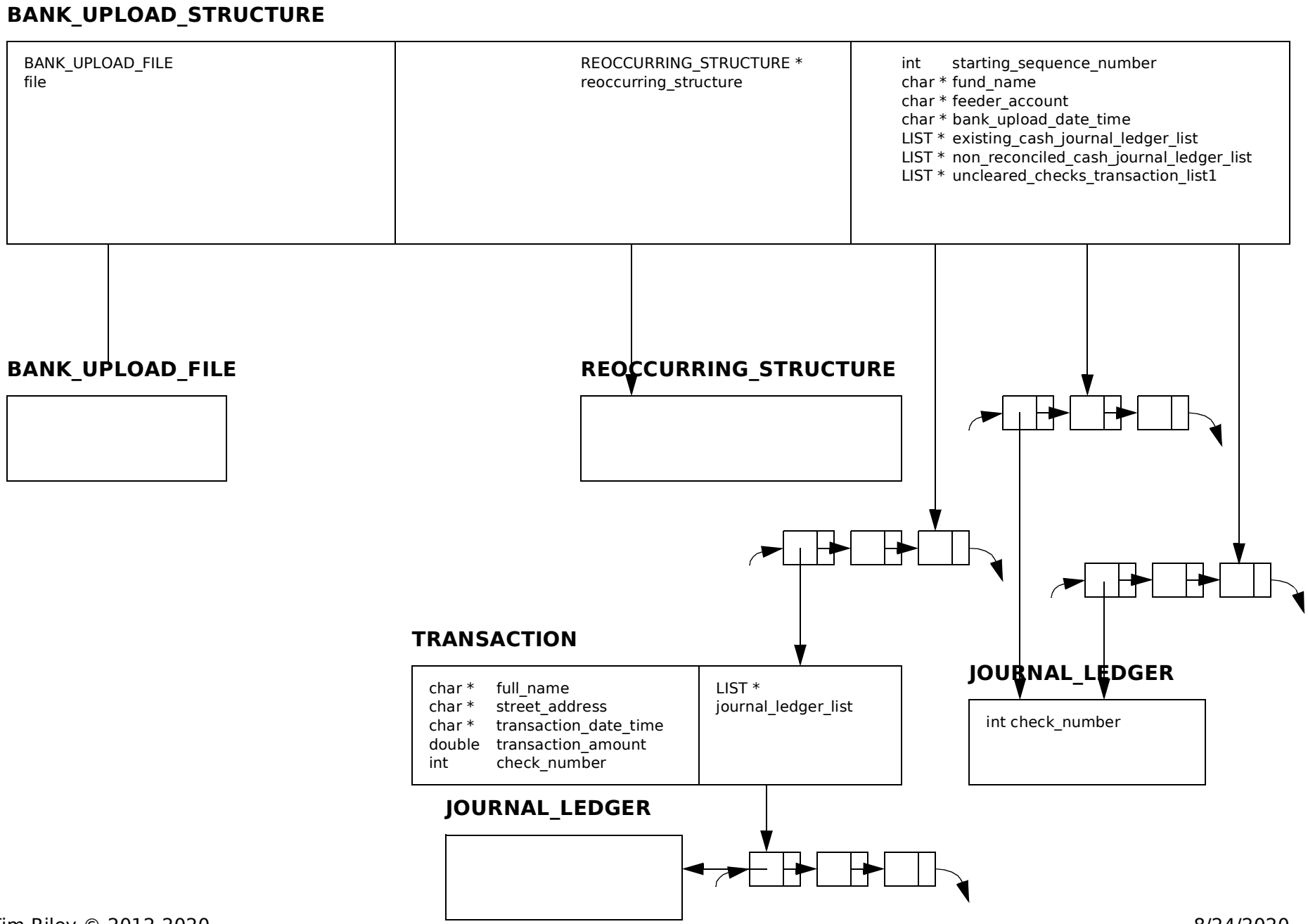


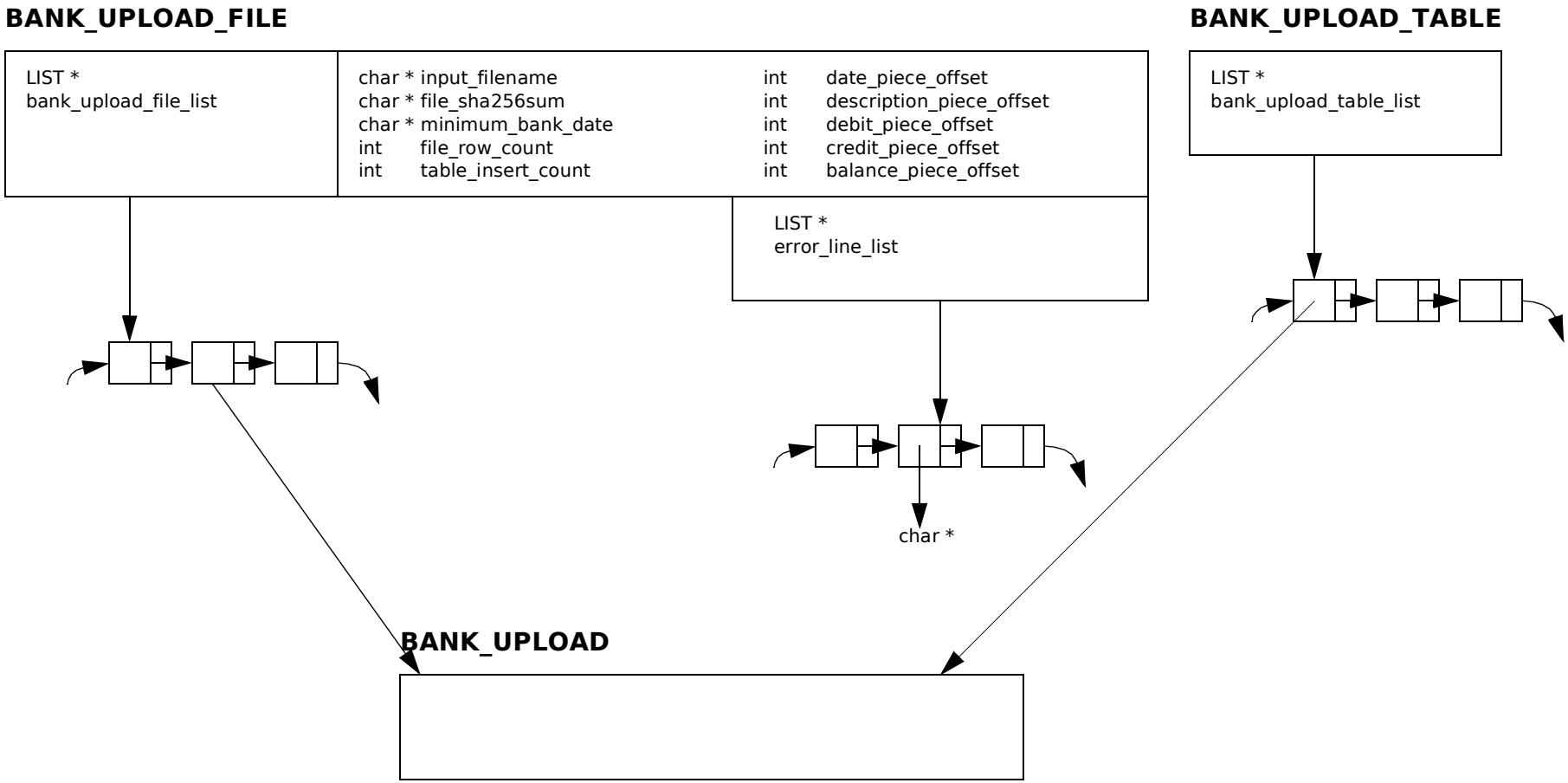

PAYROLL_POSTING
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extgension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
LIST *
employee_work_period_list
EMPLOYEE_WORK_PERIOD


EMPLOYEE_TAX_WITHHOLDING_TABLE
LIST *
federal_marital_status_list
LIST *
state_marital_status_list
LIST *
state_standard_deduction_list
EMPLOYEE_MARITAL_STATUS_WITHHOLDING
char *
marital_status
LIST *
income_tax_withholding_list
EMPLOYEE_STATE_STANDARD_DEDUCTION
char *state_marital_status
int state_withholding_allowances
double state_standard_deduction_amount
EMPLOYEE_INCOME_TAX_WITHHOLDING
int income_over
int income_not_over
double tax_fixed_amount
double tax_percentage_amount

PredictiveBooks Data Structure

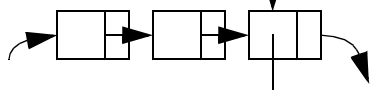



BANK_UPLOAD_STRUCTURE
BANK_UPLOAD_FILE
file
REOCCURRING_STRUCTURE *
reoccurring_structure
int starting_sequence_number
char * fund_name
char * feeder_account
char * bank_upload_date_time
LIST * existing_cash_journal_ledger_list
LIST * non_reconciled_cash_journal_ledger_list
LIST * uncleared_checks_transaction_list1
BANK_UPLOAD_FILE
REOCCURRING_STRUCTURE
TRANSACTION
char * full_name
char * street_address
char * transaction_date_time
double transaction_amount
int check_number
LIST *
journal_ledger_list
JOURNAL_LEDGER
int check_number
JOURNAL_LEDGER


BANK_UPLOAD_FILE
LIST *
bank_upload_file_list
char * input_filename
char * file_sha256sum
char * minimum_bank_date
int file_row_count
int table_insert_count
int date_piece_offset
int description_piece_offset
int debit_piece_offset
int credit_piece_offset
int balance_piece_offset
LIST *
error_line_list
char *
BANK_UPLOAD_TABLE
LIST *
bank_upload_table_list
BANK_UPLOAD

BANK_UPLOAD

char *bank_date char *bank_description int sequence_number double bank_amount double bank_running_balance	char *fund_name int check_number boolean existing_bank_upload boolean existing_transaction	LIST * reconciled_transaction_list	JOURNAL_LEDGER * cleared_journal_ledger

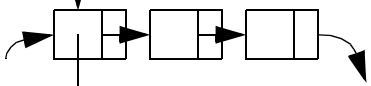


TRANSACTION

JOURNAL_LEDGER

BANK_UPLOAD

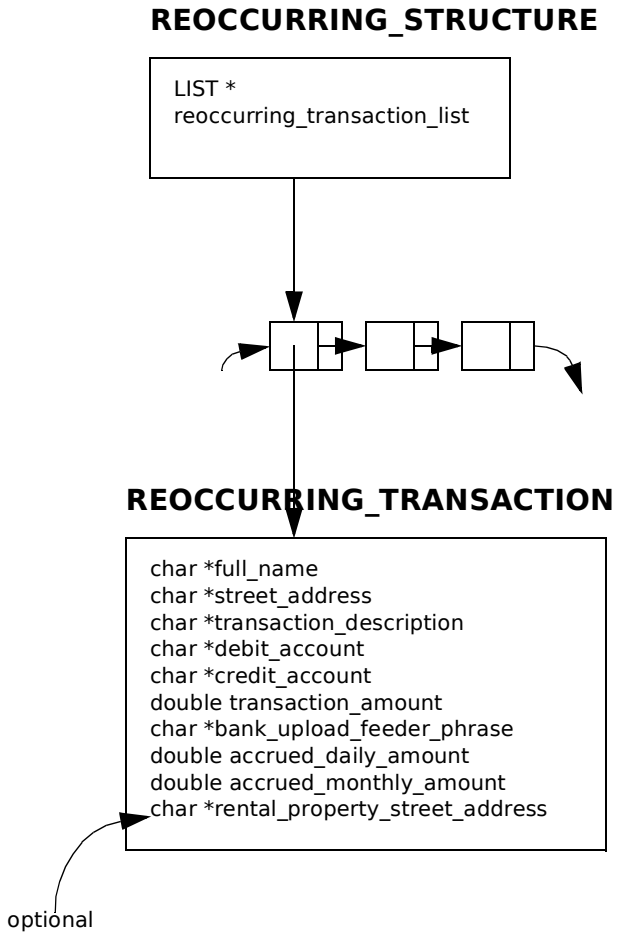
JOURNAL_LEDGER * feeder_check_number_existing_journal_ledger feeder_check_number_existing_journal_ledger(LIST *existing_cash_journal_ledger_list, LIST *uncleared_checks_transaction_list, int check_number)	LIST * feeder_match_sum_existing_journal_ledger_list feeder_match_sum_existing_journal_ledger_list(LIST *existing_cash_journal_ledger_list, double abs_bank_amount, boolean check_debit)	TRANSACTION * feeder_phrase_match_build_transaction feeder_phrase_match_build_transaction(LIST *reoccurring_transaction_list, char *bank_date, char *bank_description_embedded, double abs_bank_amount)



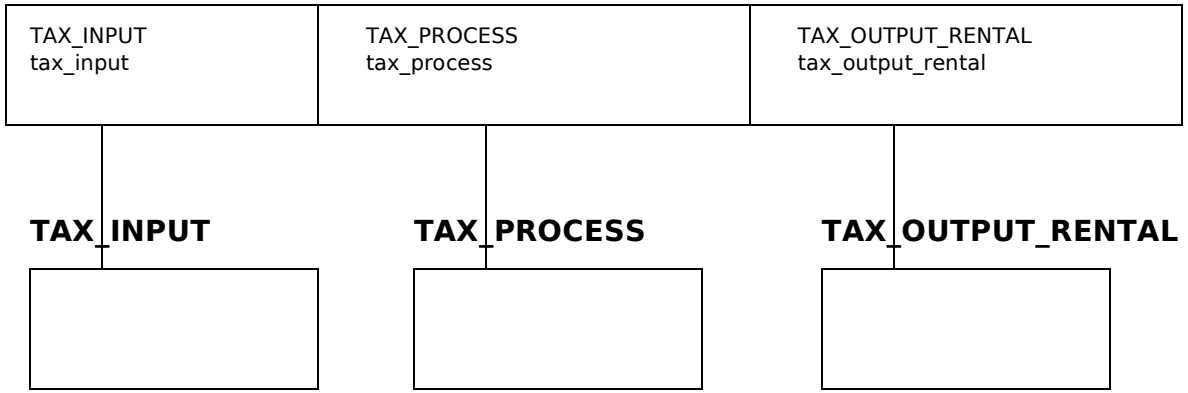
JOURNAL_LEDGER

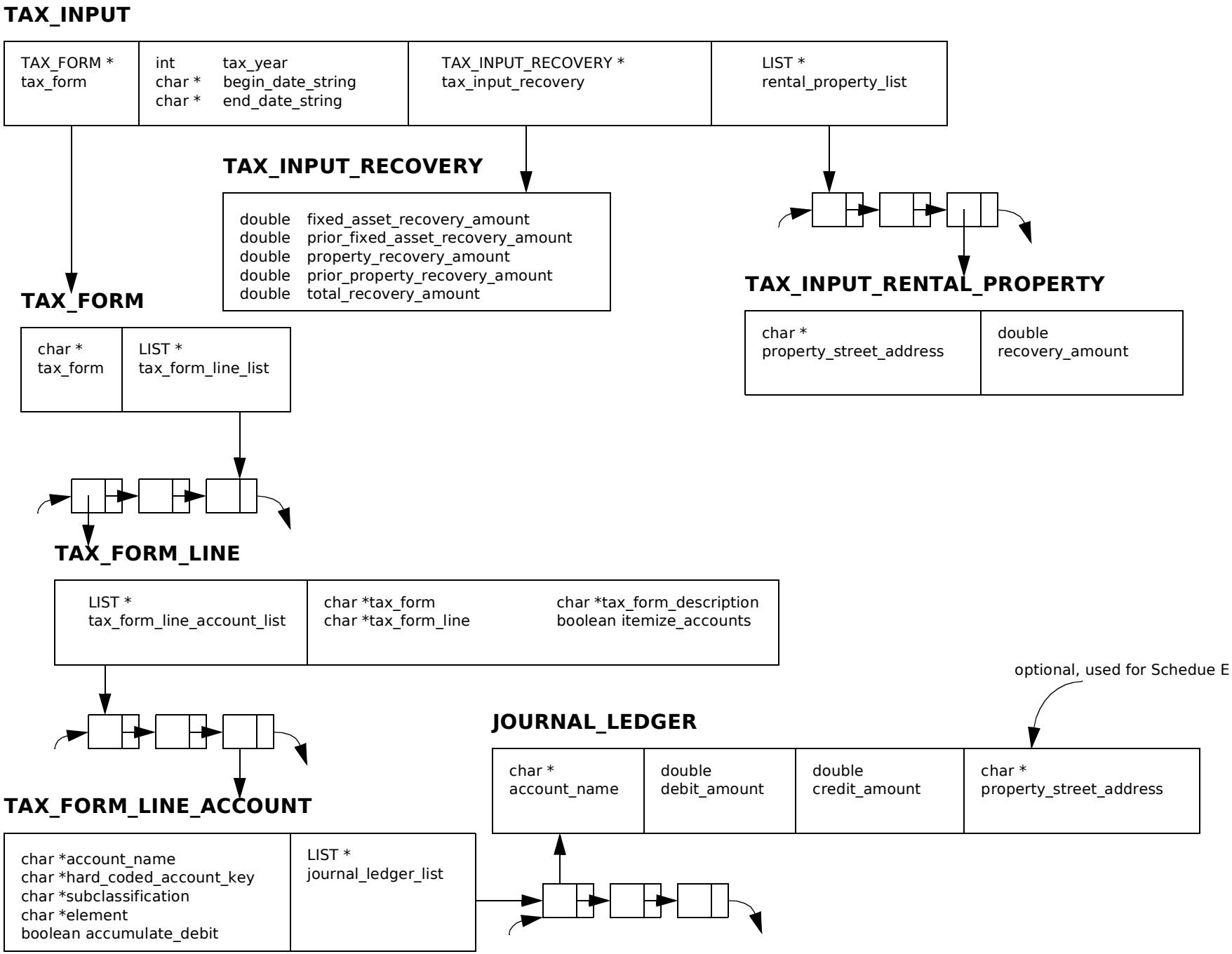
JOURNAL_LEDGER

TRANSACTION

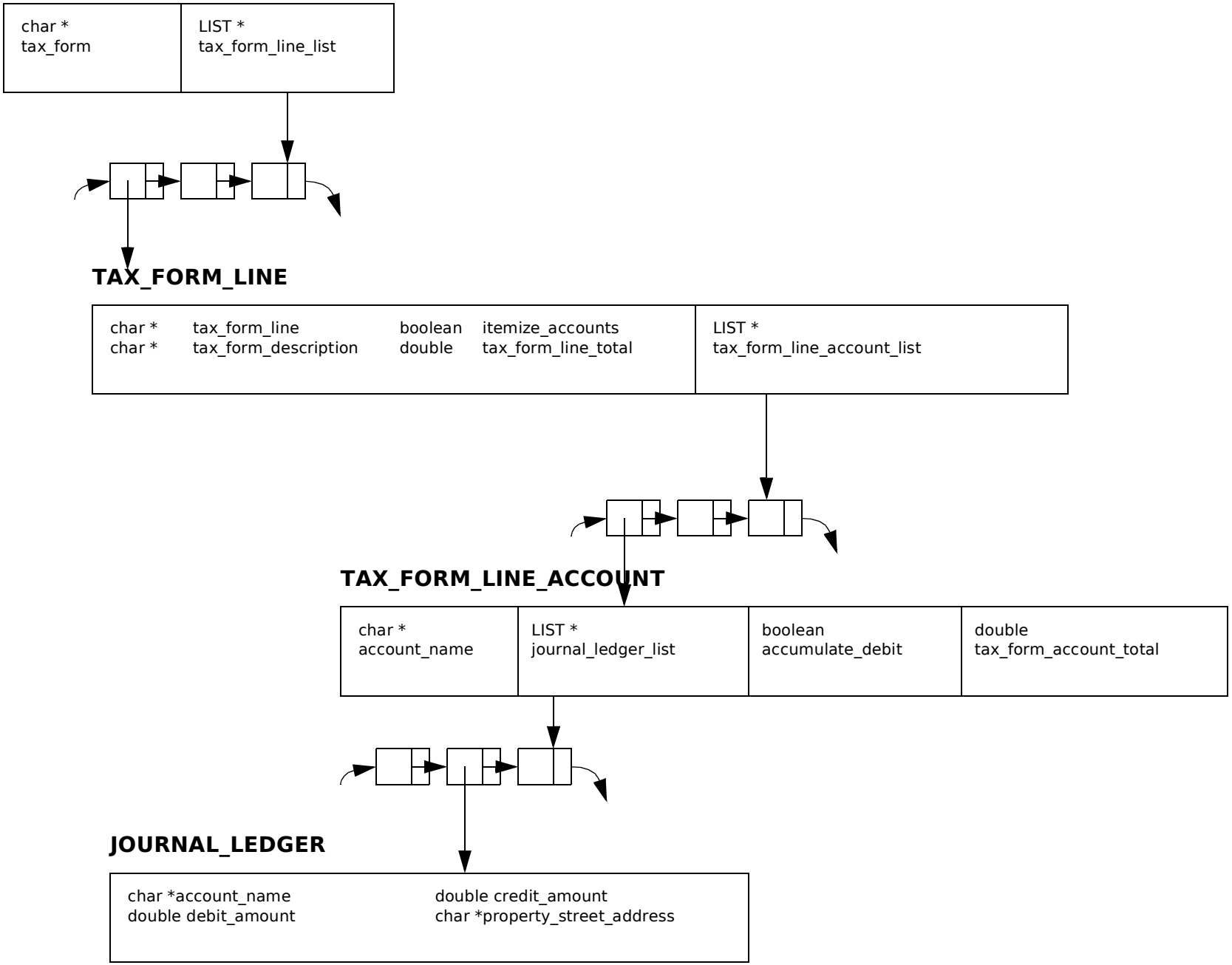

REOCCURRING_STRUCTURE
LIST *
reoccurring_transaction_list
REOCCURRING_TRANSACTION
char *full_name
char *street_address
char *transaction_description
char *debit_account
char *credit_account
double transaction_amount
char *bank_upload_feeder_phrase
double accrued_daily_amount
double accrued_monthly_amount
char *rental_property_street_address
optional

TAX

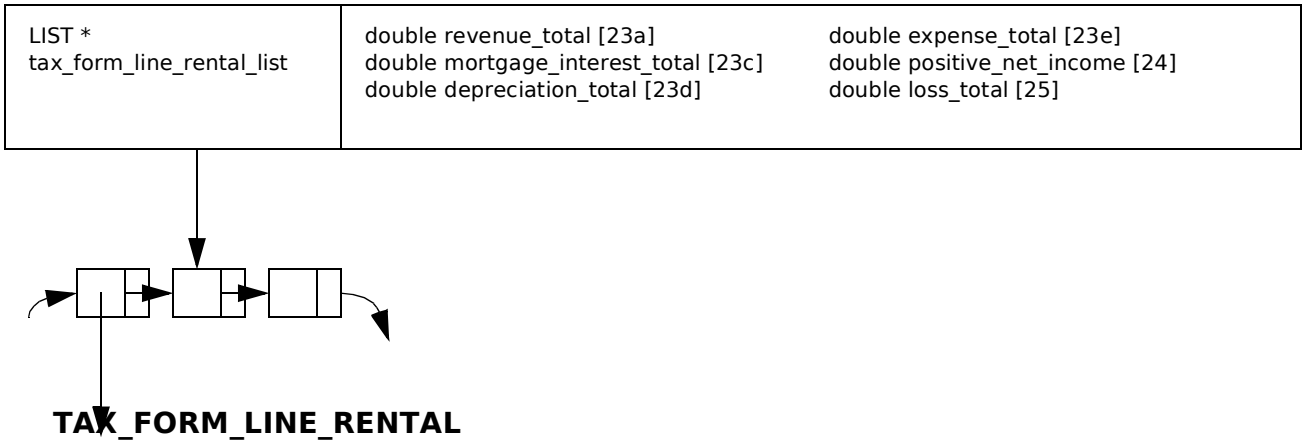

TAX_INPUT
tax_input
TAX_PROCESS
tax_process
TAX_OUTPUT_RENTAL
tax_output_rental
TAX_INPUT
TAX_PROCESS
TAX_OUTPUT_RENTAL


TAX_INPUT
TAX_FORM *
tax_form
int tax_year
char * begin_date_string
char * end_date_string
TAX_INPUT_RECOVERY *
tax_input_recovery
LIST *
rental_property_list
TAX_INPUT_RECOVERY
double fixed_asset_recovery_amount
double prior_fixed_asset_recovery_amount
double property_recovery_amount
double prior_property_recovery_amount
double total_recovery_amount
TAX_INPUT_RENTAL_PROPERTY
char *
property_street_address
double
recovery_amount
TAX_FORM
char *
tax_form
LIST *
tax_form_line_list
TAX_FORM_LINE
LIST *
tax_form_line_account_list
char *tax_form
char *tax_form_line
char *tax_form_description
boolean itemize_accounts
optional, used for Schedue E
JOURNAL_LEDGER
char *
account_name
double
debit_amount
double
credit_amount
char *
property_street_address
TAX_FORM_LINE_ACCOUNT
char *account_name
char *hard_coded_account_key
char *subclassification
char *element
boolean accumulate_debit
LIST *
journal_ledger_list

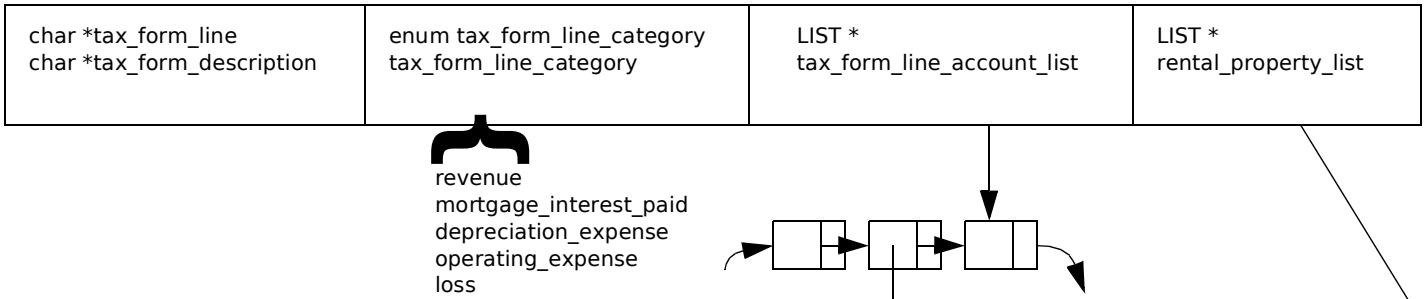
TAX_PROCESS


char *
tax_form
LIST *
tax_form_line_list
TAX_FORM_LINE
char *
tax_form_line
boolean
itemize_accounts
char *
tax_form_description
double
tax_form_line_total
LIST *
tax_form_line_account_list
TAX_FORM_LINE_ACCOUNT
char *
account_name
LIST *
journal_ledger_list
boolean
accumulate_debit
double
tax_form_account_total
JOURNAL_LEDGER
char *account_name
double credit_amount
double debit_amount
char *property_street_address

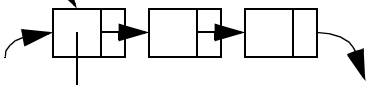
TAX_OUTPUT_RENTAL

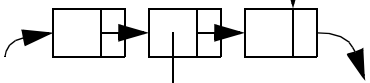


TAX_FORM_LINE_RENTAL

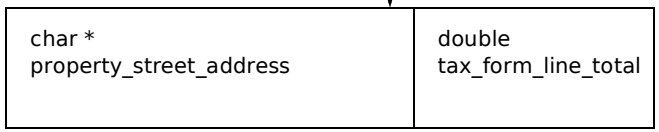


TAX_FORM_LINE_ACCOUNT

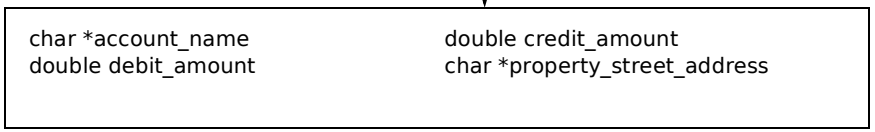




TAX_OUTPUT_RENTAL_PROPERTY



JOURNAL_LEDGER



INVESTMENT_EQUITY

INVESTMENT_INPUT investment_input	INVESTMENT_PROCESS * investment_process	INVESTMENT_ACCOUNT investment_account	LIST * input_account_balance_list	LIST * output_account_balance_list

INVESTMENT_ACCOUNT

```
char *full_name
char *street_adress
char *account_number
char *investment_account
char *fair_value_adjustment_account
char *classification
```

INVESTMENT_PROCESS

```
enum preupdate_change_state
      full_name_change_state
      street_address_change_state
      account_number_change_state
      date_time_change_state
char *earlier_date_time
char *begin_date_time
```

INVESTMENT_INPUT

```
char *date_time
char *fund_name
char *state
char *preupdate_full_name
char *preupdate_street_address
char *preupdate_account_number
char *preupdate_date_time
```

TRANSACTION

ACCOUNT_BALANCE

char *date_time char *investment_operation double share_price doubel share_quantity_change double share_quantity_balance double cash_in double market_value double book_value_change double book_value_balance	double moving_share_price double unrealized_gain_balance double unrealized_gain_change double reaqlized_gain boolean table_first_row char *transaction_date_time	TRANSACTION * transaction

PredictiveBooks Data Structure

DEPRECIATION_STRUCTURE

DEPRECIATION_DATE * depreciation_date	DEPRECIATION_TAX_YEAR * depreciation_tax_year	LIST *depreciation_fund_list

DEPRECIATION_FUND

char * fund_name char * depreciation_expense_account char * accumulated_depreciation_account		DEPRECIATION_TRANSACTION * depreciation_transaction	DEPRECIATION_ASSET_LIST * depreciation_asset_list
DEPRECIATION_AMOUNT * depreciation_amount			

DEPRECIATION_AMOUNT

double	purchase_fixed_asset_depreciation_amount
double	prior_fixed_asset_depreciation_amount
double	purchase_property_depreciation_amount
double	prior_property_depreciation_amount

DEPRECIATION_TRANSACTION

TRANSACTION *purchase_equipment_transaction
TRANSACTION *prior_fixed_asset_transaction
TRANSACTION *purchase_property_transaction
TRANSACTION *prior_property_transaction

DEPRECIATION_TAX_YEAR

int	undo_fixed_asset_year
int	undo_fixed_prior_year
int	undo_property_year
int	undo_property_prior_year
int	max_undo_year

DEPRECIATION_ASSET_LIST

LIST * purchase_fixed_asset_list	LIST * prior_fixed_asset_list	LIST * purchase_property_list	LIST * prior_property_list

DEPRECIATION_DATE

char *	undo_fixed_asset_date
char *	prior_fixed_asset_date
char *	undo_fixed_prior_date
char *	prior_fixed_prior_date
char *	undo_property_date
char *	prior_property_date
char *	undo_property_prior_date
char *	prior_property_prior_date
char *	max_undo_date

EQUIPMENT_PURCHASE